Лабораторная работа №3

Агентное моделирование.

Талебу Тенке

Российский университет дружбы народов, Москва, Россия

::: :::::::::::::
::: :::::::::::::

Цель работы

Агентный подход к имитационному моделированию (Agent-Based Modeling, ABM) — это метод исследования сложных систем, в котором поведение системы возникает из взаимодействия множества автономных сущностей, называемых агентами. Вместо того чтобы описывать систему глобальными уравнениями, мы моделируем каждую индивидуальную единицу и правила её поведения, а затем наблюдаем, какие коллективные паттерны появляются снизу вверх. Этот подход особенно полезен, когда поведение системы трудно предсказать из-за нелинейностей, гетерогенности участников или адаптивных стратегий.

Задание

1. Создать рабочий каталог для кода.
2. Установить необходимые пакеты.
3. Выполнить предложенный код.
4. Преобразовать код в литературный стиль.
5. Сгенерировать из литературного кода:

- чистый код;
- jupyter notebook;
- документацию в формате Quarto.

6. Выполнить код из jupyter notebook.
7. Интегрировать документацию в формате Quarto в отчёт.
8. Добавить в код в литературном стиле вычисление для набора параметров.
9. Сгенерировать из литературного кода с параметрами:

- чистый код;

Выполнение лабораторной работы

Реализация на Agents.jl

- Мы создадим файл src/daisyworld.jl. Здесь мы определим тип агента и функции шага модели.

```
julia> include("src/daisyworld.jl")
daisyworld (generic function with 1 method)

julia> model = daisyworld()
StandardABM with 360 agents of type Daisy
 agents container: Dict
 space: GridSpaceSingle with size (30, 30), metric=chebyshev, periodic=true
 scheduler: fastest
 properties: temperature, solar_luminosity, max_age, surface_albedo, ratio, solar_change, tick, scenario

julia> step!(model, 50)
StandardABM with 894 agents of type Daisy
 agents container: Dict
 space: GridSpaceSingle with size (30, 30), metric=chebyshev, periodic=true
 scheduler: fastest
 properties: temperature, solar_luminosity, max_age, surface_albedo, ratio, solar_change, tick, scenario

julia>
```


Рис. 1: Рис

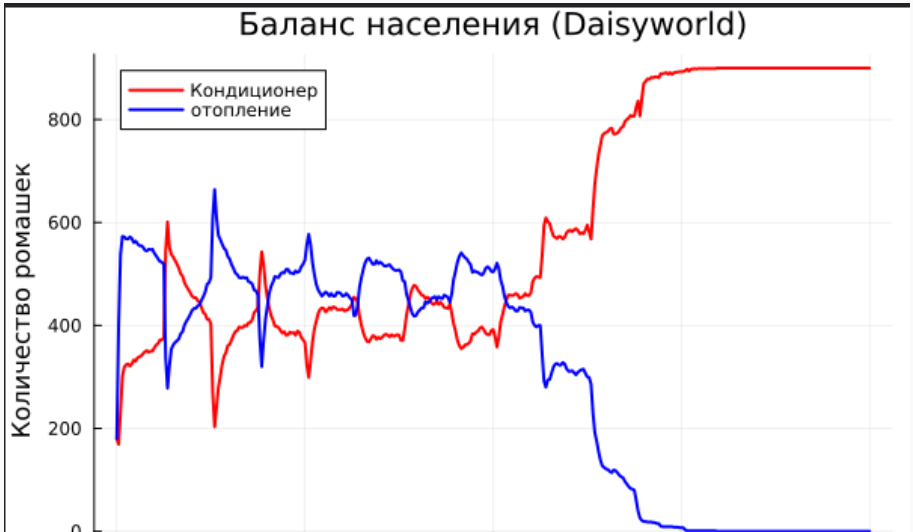
Реализация на Agents.jl

- Таблица, которая мы получили.

```
julia> model = daisyworld(scenario = :ramp)
StandardABM with 360 agents of type Daisy
 agents container: Dict
 space: GridSpaceSingle with size (30, 30), metric=chebyshev, periodic=true
 scheduler: fastest
 properties: temperature, solar_luminosity, max_age, surface_albedo, ratio, solar_change, tick, scenario

julia> df_agents, df_model = run!(model, 400; adata, mdata)
(401×3 DataFrame
 Row │ time   count_white  count_black
     │ Int64  Int64        Int64
─────┼──────────────────────────────────
   1 │     0          180          180
   2 │     1          169          400
   3 │     2          233          538
   4 │     3          301          573
   5 │     4          319          572
   6 │     5          324          568
   7 │     6          325          568
   8 │     7          321          572
   9 │     8          325          569
  10 │     9          333          561
  11 │    10          330          564
  12 │    11          337          559
  13 │    12          339          555
  14 │    13          341          555
  15 │    14          345          552
  16 │    15          349          546
   ⋮ │     ⋮          ⋮            ⋮
 386 │   385          900            0
 387 │   386          900            0
 388 │   387          900            0
 389 │   388          900            0
```

- График баланса населении.



Базовая визуализация

- Анализ полученной визуализации показывает неоднородное распределение температуры по поверхности модели. Можно заметить корреляцию между плотностью популяции маргариток и локальной температурой: черные маргаритки (с низким альбедо) способствуют поглощению тепла, в то время как белые маргаритки (с высоким альбедо) отражают солнечное излучение. Данная визуализация является отправной точкой для исследования механизмов саморегуляции климата в модели.

```
using DrWatson
@quickactivate
using Agents
using DataFrames
using Plots

include(srcdir("daisyworld.jl"))

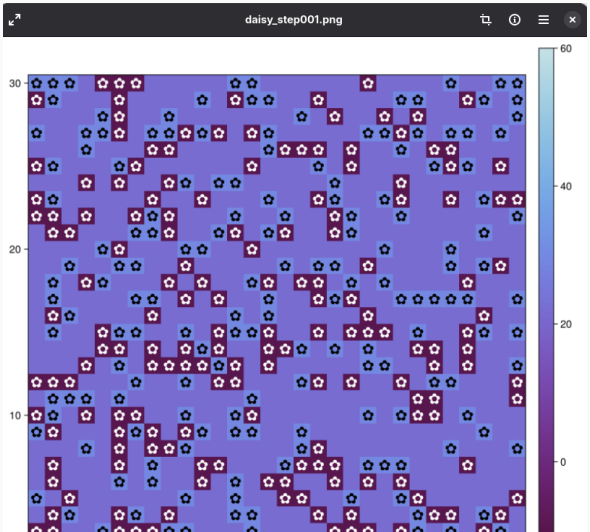
using CairoMakie
model = daisyworld()

daisycolor(a::Daisy) = a.breed

plotkwargs = (
    agent_color=daisycolor, agent_size = 20, agent_marker = '✿',
    heatarray = :temperature,
```

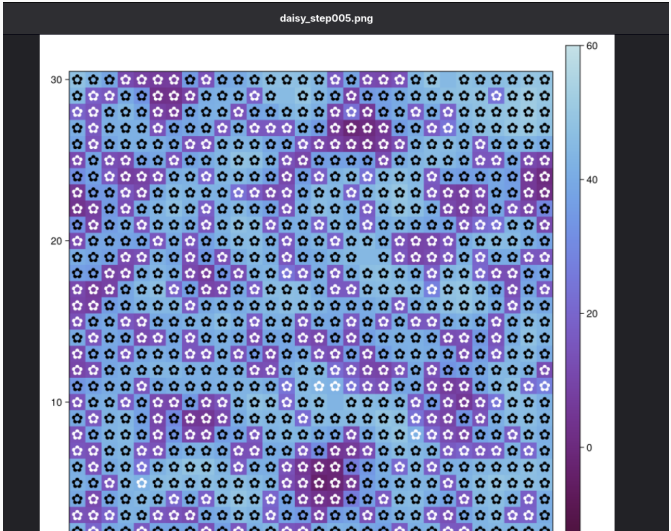
Базовая визуализация

- Визуализация 1:



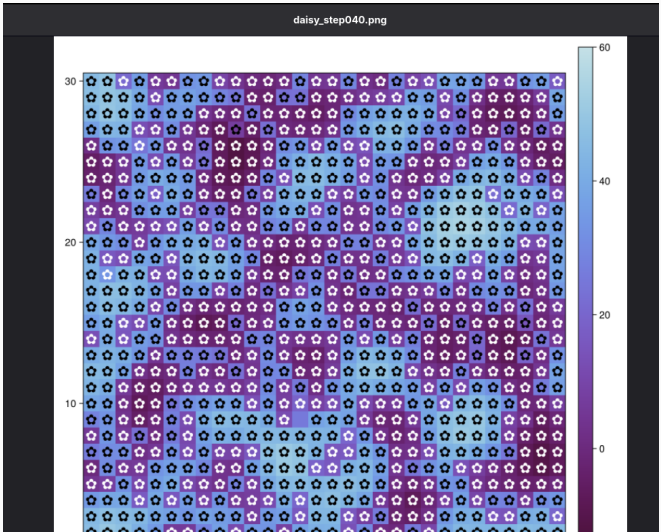
Базовая визуализация

- Визуализация 2:



Базовая визуализация

- Визуализация 3:



- Создание файла scripts/daisyworld-animate.jl.

```
julia> touch("scripts/daisyworld-animate.jl")
"scripts/daisyworld-animate.jl"

julia> include("scripts/daisyworld-animate.jl")
  Activating project at `~/work/study/2025-2026/simulation-modeling/study_2025-2026_simulation-modeling/l
abs/lab03`
```

Рис. 8: Рис

Анимация модели

- Анимация модели позволила выявить фазы экспансии и сокращения популяций в зависимости от локальных температурных условий. На видео четко прослеживается эффект “обратной связи”: черные маргаритки начинают доминировать при низких температурах, разогревая поверхность, что создает условия для последующего распространения белых маргариток. Этот динамический процесс является ключевым доказательством устойчивости моделируемой экосистемы.

- Код

Ouvrir

daisyworld-animate.jl

~/work/study/2025-2026/simulation-modeling/study_2025-2026_simulation-modeling/labs/lab03/scripts

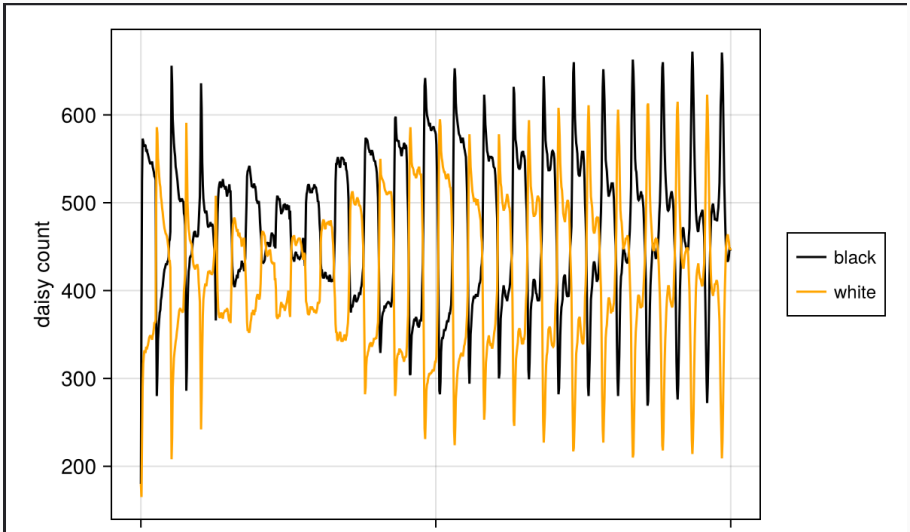
```
using DrWatson
@quickactivate
using Agents
using DataFrames
using Plots

include(srcdir("daisyworld.jl"))

using CairoMakie
model = daisyworld()
```

Динамика числа маргариток

- Построим график изменения числа маргариток в зависимости от модельного времени



Динамика числа маргариток

- Сгенерирование из литературного кода: jupyter notebook

[1]:
```
using DrWatson
@quickactivate "project"

using Agents, DataFrames, CairoMakie
include(srcdir("daisyworld.jl"))
```

[1]:
```
daisyworld (generic function with 1 method)
```

Сбор данных

Мы определяем вспомогательные функции для фильтрации и подсчета агентов в зависимости от их типа (породы).

[2]:
```
black(a) = a.breed == :black
white(a) = a.breed == :white
```

[2]:
```
white (generic function with 1 method)
```

Настраиваем сбор данных: на каждом шаге будет подсчитываться количество черных и белых маргариток.

[3]:
```
adata = [(black, count), (white, count)]
```

[3]:
```
2-element Vector{Tuple{Function, typeof(count)}}:
 (Main.var"##230".black, count)
 (Main.var"##230".white, count)
```

Динамика числа маргариток

- Таблица

jupyter daisy_count_notebook Last Checkpoint: 2 minutes ago

File Edit View Run Kernel Settings Help Not Trusted

Markdown JupyterLab Julia 1.10

[4]:
```
model = daisyworld(; solar_luminosity = 1.0)
agent_df, model_df = run!(model, 1000; adata)
```

[4]: (1001×3 DataFrame

Row	time	count_black	count_white
	Int64	Int64	Int64
1	0	180	180
2	1	383	169
3	2	529	232
4	3	563	293
5	4	563	324
6	5	560	336
7	6	556	340
8	7	557	339
⋮	⋮	⋮	⋮
995	994	486	412
996	995	487	411
997	996	487	410
998	997	485	413
999	998	487	410
1000	999	486	412
1001	1000	488	410

986 rows omitted, **0×0 DataFrame**)

Визуализация

Создаем график динамики популяций, аналогичный рисунку 3.2 из лабораторной работы.

[5]:
```
figure = Figure(size = (600, 400))
ax = figure[1, 1] = Axis(figure, xlabel = "Время (тики)", ylabel = "Количество маргариток")
```

[5]: Makie.Axis with 0 plots:

Строим линии для черных и белых маргариток.

Динамика числа маргариток

- Визуализации

[6]:
```
blackl = lines!(ax, agent_df[!, :time], agent_df[!, :count_black], color = :black)
whitel = lines!(ax, agent_df[!, :time], agent_df[!, :count_white], color = :orange)
```

[6]:
```
Makie.Lines{Tuple{Vector{GeometryBasics.Point{2, Float64}}}}
```

Добавляем легенду в правой части графика (позиция [1, 2]).

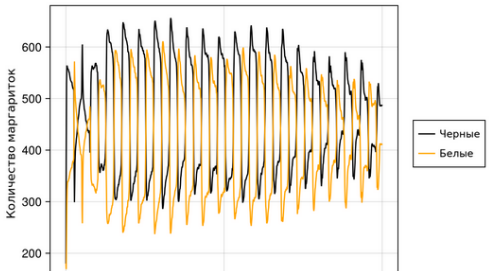
[7]:
```
Legend(figure[1, 2], [blackl, whitel], ["Черные", "Белые"], labelsize = 12)
```

[7]:
```
Makie.Legend()
```

Отображение итогового графика:

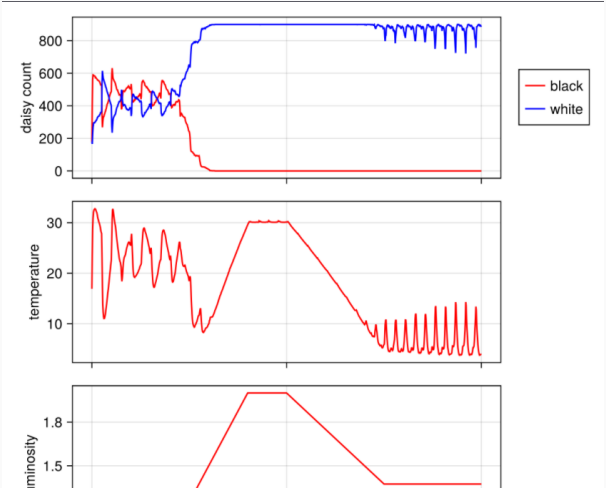
[8]:
```
figure
```

[8]:



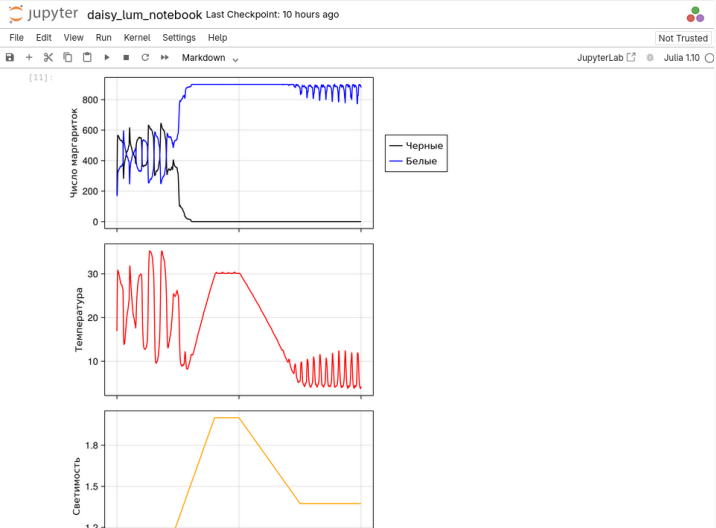
Динамика модели

- Построим комплексный график изменения числа маргариток, температуры, аль- бедо в зависимости от модельного времени



Динамика модели

- Визуализации в jupyter notebook



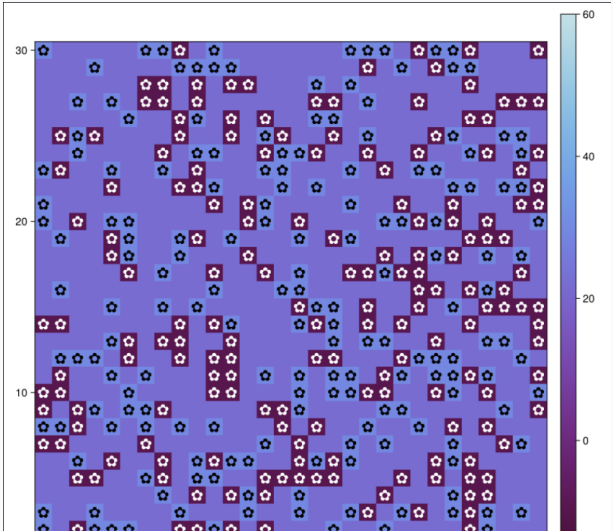
Базовая визуализация (параметры)

- В данном подразделе реализовано расширение базовой модели за счет введения варьируемых параметров эксперимента. С помощью словаря param_dict определены диапазоны значений для таких характеристик, как максимальный возраст маргариток (max_age) и начальная доля белых маргариток (init_white). Это позволяет автоматизировать процесс проведения серии симуляций (ensemble runs) и в дальнейшем проанализировать, как начальные условия влияют на стабильность климатической системы “Мира маргариток”.



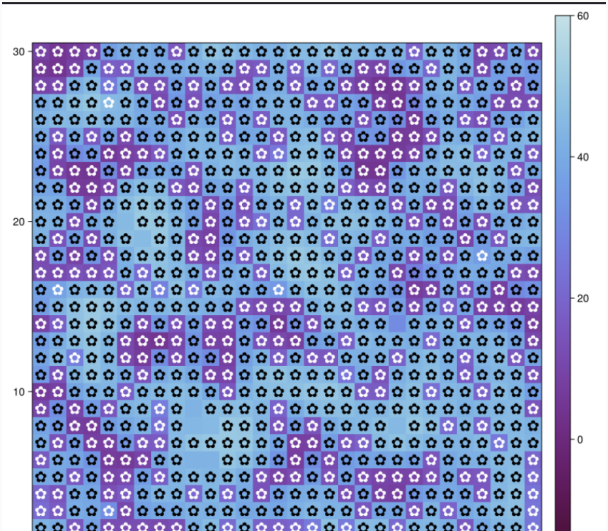
Базовая визуализация (параметры)

- Визуализации 1



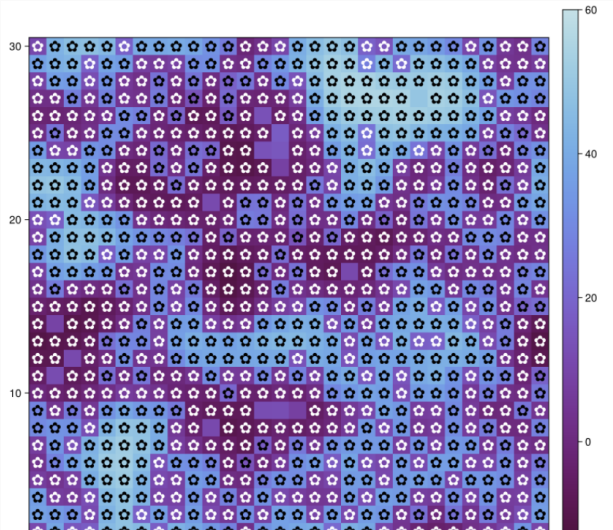
Базовая визуализация (параметры)

- Визуализации 2



Базовая визуализация (параметры)

- Визуализации 3



Динамика числа маргариток (параметры)

- Процесс генерации графиков динамики был полностью автоматизирован в рамках парадигмы литературного программирования. Код из файла scripts/daisyworld-count__param.jl интегрирован в итоговый отчет Quarto, что гарантирует соответствие представленных данных текущим настройкам параметров эксперимента. Такой подход исключает ошибки ручного переноса данных и позволяет быстро пересчитать все результаты при изменении исходных условий в словаре параметров.

Ouvrir

daisyworld-count__param.jl

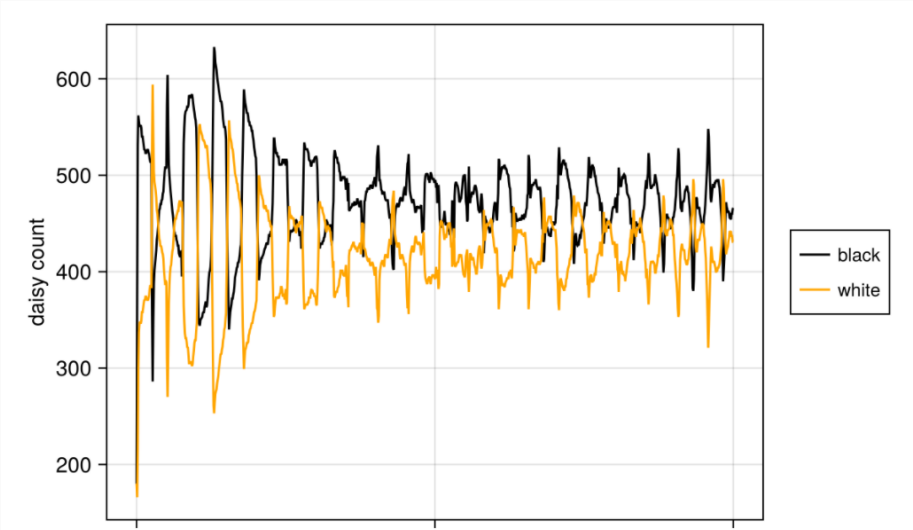
~/work/study/2025-2026/simulation-modeling/study_2025-2026_simulation-modeling/labs/lab03/scripts

```
black(a) = a.breed == :black
white(a) = a.breed == :white
adata = [(black, count), (white, count)]

## Параметры эксперимента
param_dict = Dict(
    :griddims => (30, 30),
    :max_age => [25, 40],
    :init_white => [0.2, 0.8],
    :init_black => 0.2,
    :albedo_white => 0.75,
    :albedo_black => 0.25,
    :surface_albedo => 0.4,
    :solar_change => 0.005,
    :solar_luminosity => 1.0,
    :scenario => :default,
```

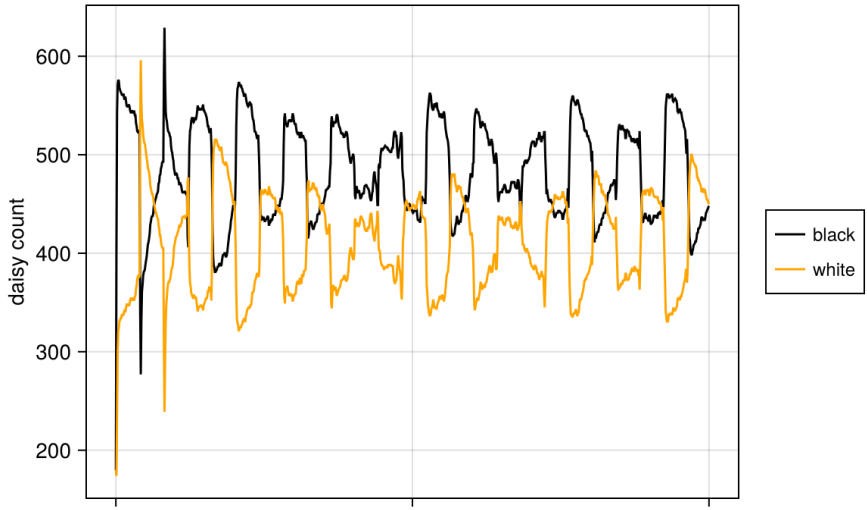
Динамика числа маргариток (параметры)

- Визуализации 1



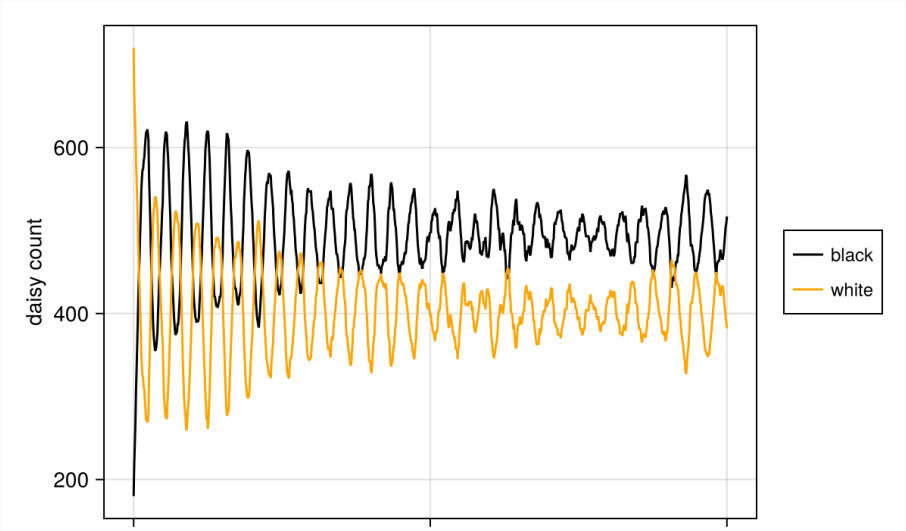
Динамика числа маргариток (параметры)

- Визуализации 2



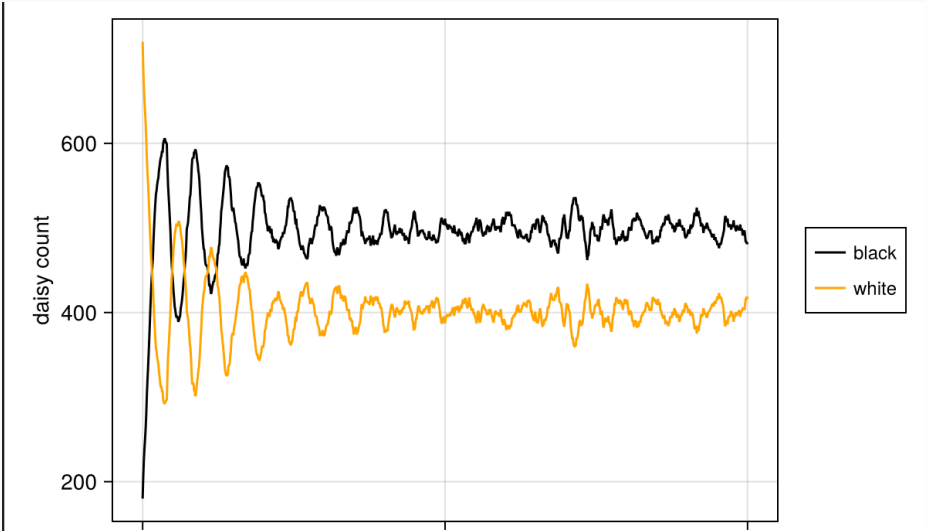
Динамика числа маргариток (параметры)

- Визуализации 3



Динамика числа маргариток (параметры)

- Визуализации 4



Динамика модели (параметры)

- Построим комплексный график изменения числа маргариток, температуры, альбедо в зависимости от модельного времени с разными параметрами модели.

Ouvrir

daisyworld-luminosity__param.jl

~/work/study/2025-2026/simulation-modeling/study_2025-2026_simulation-modeling/labs/lab03/scripts

daisyworld-count__param.jl | daisyworld-luminosity__param.jl

```
## Создаём список всех комбинаций
params_list = dict_list(param_dict)

for params in params_list

    model = daisyworld(;params...)
    temperature(model) = StatsBase.mean(model.temperature)
    mdata = [temperature, :solar_luminosity]

    agent_df, model_df = run!(model, 1000; adata = adata, mdata = mdata)
    figure = CairoMakie.Figure(size = (600, 600));
    ax1 = figure[1, 1] = Axis(figure, ylabel = "daisy count")
    blackl = lines!(ax1, agent_df[!, :time], agent_df[!, :count_black], color = :red)
    whitel = lines!(ax1, agent_df[!, :time], agent_df[!, :count_white], color = :blue)
    figure[1, 2] = Legend(figure, [blackl, whitel], ["black", "white"])

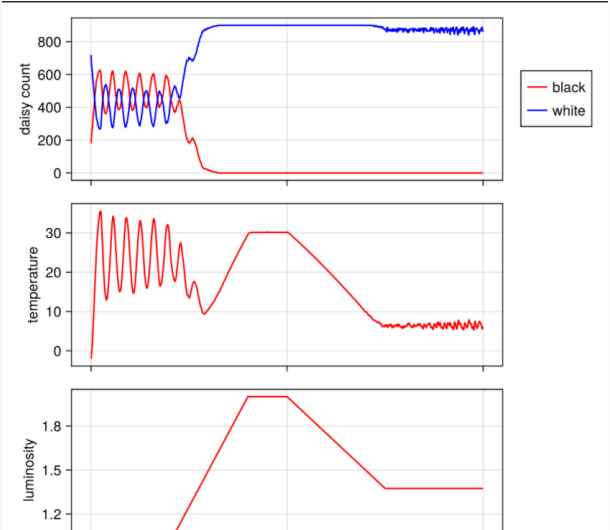
    ax2 = figure[2, 1] = Axis(figure, ylabel = "temperature")
    ax3 = figure[3, 1] = Axis(figure, xlabel = "tick", ylabel = "luminosity")
    lines!(ax2, model_df[!, :time], model_df[!, :temperature], color = :red)
    lines!(ax3, model_df[!, :time], model_df[!, :solar_luminosity], color = :red)

    for ax in (ax1, ax2); ax.xticklabelsvisible = false; end

    plt_name = savename("daisy-luminosity",params) * ".png"
    save(plotsdir(plt_name), figure)
end
```

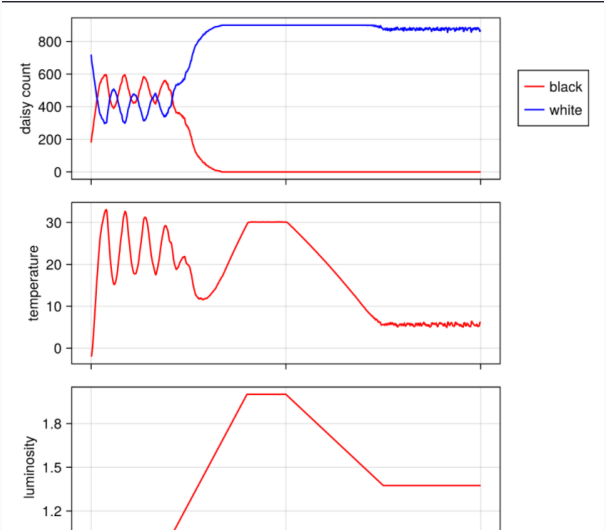
Динамика модели (параметры)

- Визуализации 1



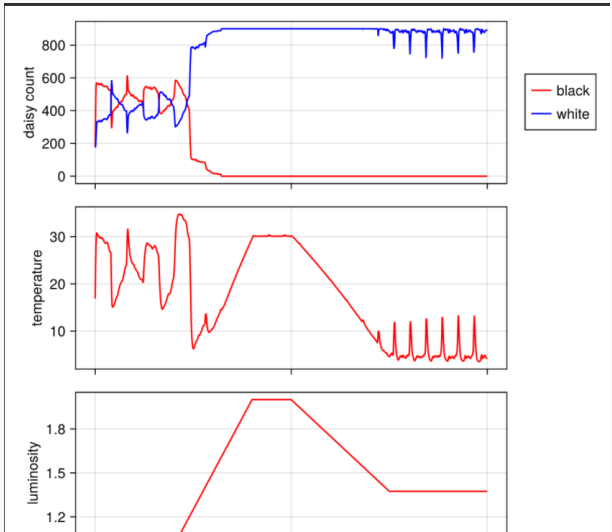
Динамика модели (параметры)

- Визуализации 2



Динамика модели (параметры)

- Визуализации 3



Выводы

- Я выполнила лабораторную работу.